

Database Learning — Master Prompts (SQL Server → MongoDB)

These prompts are built from the ones in `Master Learning Prompts/`, `Learning DSA with Claude/`, `Programmer Fundamentals Guide and Prompt/`, `Backend/DotNet_Learning_Path/prompt` and `PDF Generation Guide and Prompts/`. They are retargeted to databases, and the hands-on work runs against one practice database: **RetailOrderDb**.

Where each part came from:

Source prompt	What was kept	What was dropped / changed
Updated Master Prompt (8 sections, priority-ordered)	The base structure. Sections are ordered by how much they're worth in the PDF. Includes "stop cleanly at the end of a section".	The Angular-specific sections were replaced with DB ones.
Priority Order & Token-Efficient Workflow	The Gen / Doubt / Consolidate / PDF triggers and one module per chat.	"One topic per chat" became "one module per chat", because topics inside a module build on each other.
DSA Master Prompt	The Pattern Recognition Signal and thinking-process sections. They map directly onto the operations-to-tool rule.	Brute force → optimised became naive query → correct query → proof.
Programmer Fundamentals 16-section	Anti-patterns in production, and the rule that every analogy has a name.	The 16-section length. It's too heavy for 50+ topics.
.NET Prompt X / Y	Scope discipline, specific notes on where a concept shows up in real code, the PDF spec, and the 86-char code-line validation.	—
Updated SEEBA	The "find the key underlying mechanism first" rule, retargeted to DB mechanisms.	JS execution-context rules.
master-prompts.md (build project)	The "guide me, don't solve it" style for labs.	Study-plan prompt. The roadmap now exists.
claude-learning-prompts.pdf / GPT prompts	A few situational prompts, adapted (see the end of this file).	Angular-specific ones.
Database/.../Prompt (roadmap generator)	Nothing. It was a one-off that produced the old roadmap, and the new roadmap replaces it.	—

0. Trigger cheat sheet

You type	What happens
Gen: 3.2	Full topic response for roadmap topic 3.2 (Prompt A)
Gen lite: 4.4	Short response for KNOW topics (Prompt A, lite mode)
SEEBA - [concept]	Quick concept answer (Prompt B)
Doubt: 3.2 S4 – [gap]	Fixes only that gap in section 4 of topic 3.2
Lab: M3	Module scenario pack: realistic tickets, no solutions (Prompt C)
Review: M3-T2 + your SQL	Senior-DBA code review of your answer (Prompt D)
Reveal: M3-T2	Reference solution, only after you've tried it
Drill: M3	Rapid recognition drill: plain-English requirement → tool (Prompt E)
Interview: M3	Mock interview round, one question at a time with feedback
Incident: [symptom]	Guided production-debugging walkthrough. You drive; Claude asks what you'd check next.
Consolidate: 3.2	Cleans up the topic and folds in the clarifications from doubts (Prompt F)
Module PDF: M3	Builds the module's reference PDF (Prompt G)
Status	Reads <code>learning-progress.md</code> and says where you are and what's next

Workflow per module (one chat per module):

1. Gen: each topic in order. Read it yourself, then run its section 8 lab tasks in SSMS.
 2. Doubt: only for specific gaps. Keep each doubt to 2 lines or fewer, and name the section number.
 3. Lab: Mx, then Review: for each ticket.
 4. Drill: Mx. This is the recognition check.
 5. Consolidate: each topic, then Module PDF: Mx. Commit the PDF and your lab `.sql` files to the repo.
 6. Update `learning-progress.md`.
-

Prompt A — Topic Generation (Gen: [topic id])

DATABASE LEARNING SERIES – PHASE 1: SQL SERVER / T-SQL

Teach me roadmap topic [TOPIC ID – TOPIC NAME] from SQL_Server_Roadmap.md.

MY PROFILE

- Full-stack developer, 5 years Angular + .NET. I know basic SELECT/INSERT/CREATE. I write data access through EF Core day to day.
- Goal: write correct, fast SQL for real apps, handle the common production scenarios, and be interview-ready for senior full-stack roles.
- Learning style: named analogy → the mechanism → code → hands-on. Depth over speed. I want the WHY, and I want to recognise WHEN to use it from a plain-English requirement.

PRACTICE ENVIRONMENT

- SQL Server (Developer Edition) + SSMS. Every example and lab runs against RetailOrderDb (schema in the roadmap: Customers, Addresses, Categories, Products, Orders, OrderItems, Payments, Employees), seeded with realistic volume (~100k orders).
- Use SET STATISTICS IO, TIME ON and the actual execution plan as the feedback loop whenever performance is involved.

RULES

1. Scope discipline: cover exactly this topic. If a prerequisite is missing, flag it in one line with its topic id; don't teach it inline.
2. Name every analogy. Before explaining, identify the KEY UNDERLYING MECHANISM that makes this topic click (e.g. logical processing order, three-valued logic, B-tree pages, locks vs row versions, the plan cache) and build the explanation around it.
3. All T-SQL must be valid on SQL Server 2019+, run against RetailOrderDb, and be commented on every non-obvious line. Flag any 2022+ only syntax.
4. Code lines ≤ 86 characters (PDF constraint).
5. Stop cleanly at the end of a section if the response gets long. Say "Stopped after Section X – say 'continue'". Never cut a section in half.
6. No preamble, no closing summary.

FORMAT – 11 SECTIONS, IN THIS ORDER

0. Rating

- Difficulty: Beginner / Intermediate / Advanced (one-line reason)
- App-dev relevance: Medium / High / Critical (one-line reason)
- Interview relevance: Medium / High / Critical (one-line reason)

1. The Problem It Solves

- Named analogy (real-life, not programming).
- The key underlying mechanism, in plain English.
- The class of real problems this fixes, and the ones it does NOT fix.
- Comparison table if the topic has variants.

2. Recognition Signals (operations → tool)

- 4–6 plain-English requirement phrases (how a PM, ticket or interviewer

says it) that should trigger this tool, e.g. "latest order per customer" → ROW_NUMBER / APPLY.

- The look-alike tool people wrongly reach for, and how to tell them apart.

3. Core Syntax & Mechanics

- Minimal commented T-SQL isolating the concept on RetailOrderDb.
- For performance topics: the before/after plan shape and logical reads I should expect to see.

4. Scenario Walkthrough

- One realistic ticket in plain English.
- Thinking-out-loud monologue: "The ticket says X... so the grain is Y... which means Z..."
- Naive query → what's wrong with it (wrong result or slow) → correct query → a verification query that proves it's right (row counts, reconciliation total, or plan/I/O comparison).

5. Wrong Tool & Tradeoffs

- When this is the WRONG choice, with the concrete condition.
- The alternative, and the exact condition under which it wins.
- Cost this adds (writes, storage, locking, complexity), if any.

6. Production Gotchas (exactly 3)

- Each: symptom seen in prod → cause → fix → how to detect it early.

7. .NET / EF Core Bridge

- The LINQ / EF Core equivalent (or why EF can't express it → raw SQL).
- The SQL EF actually generates, and the pitfall hiding there.
- Keep it short; skip with one line if genuinely not applicable.

8. Hands-on Lab (no solutions)

- 3 tasks against RetailOrderDb: Warm-up → Real ticket → Stretch.
- Each: the requirement written as a ticket + a self-check (expected row count, a reconciliation total, or the plan shape to look for).
- No hints, no solutions. I'll use Review:/Reveal: afterwards.

9. Interview Prep (2 questions)

- How it's asked → what a strong answer covers (3–5 bullets) → what a weak answer misses → the likely follow-up. Level: Mid / Senior.

10. Quick Reference Card

- Concept in one sentence.
- Syntax at a glance.
- Use when / don't use when.
- The one rule that matters most (bold).
- Memory aid (one line).

Lite mode (Gen Lite:), for KNOW topics: same header and rules, but only Sections 1, 2, 3, 5 and 10, plus **one** lab task.

Awareness sheet (Gen aware: M-aware), for AWARE topics: for each topic, give 3 lines covering what it is, when you'd meet it, and who usually owns it.

Prompt B — SEEBA for databases (SEEBA - [concept])

This is used for quick doubts and concept checks that aren't a full topic.

- S – One-line summary (bold, plain English)
- E – Simple explanation, 2–3 lines, conversational, build-something style.
Lead with the key underlying mechanism (processing order, 3-valued logic, B-tree/pages, locks vs versions, plan cache/statistics).
- E – Example: commented T-SQL against RetailOrderDb first
(MongoDB shell/aggregation syntax first for Phase 2 topics)
- B – 3–4 bullet breakdown
- A – Applied relevance: 2–3 lines on where it shows up in real full-stack work
(EF Core / .NET data layer; Node or .NET driver for MongoDB) + short snippet

Rules: short, no deep detail unless asked. If follow-ups keep coming on the same concept, switch to a completely different angle – never repeat the same explanation.

Prompt C — Module Scenario Pack (Lab: M[n])

Generate the scenario pack for module M[n] of SQL_Server_Roadmap.md.

- 5 tickets written like real Jira tickets from a PM, support engineer or another dev – plain English, no SQL keywords that give the answer away.
- Mix: 2 feature/reporting requests, 1 bug report (wrong numbers or missing rows), 1 performance complaint, 1 data-fix or edge-case ticket. Adjust the mix to what the module actually covers.
- Each ticket: ID (M[n]-Tl...), context, acceptance criteria, and a self-check (expected row count / total / plan shape) computed against the seeded RetailOrderDb. If a figure depends on random seed data, give the query shape of the check instead of a number.
- Tickets may use anything from earlier modules, never later ones.
- No hints and no solutions. Wait for my answers.

Prompt D — Review (Review: M[n]-T[k] + my SQL)

Review my solution like a senior engineer doing a code review.

1. Correctness: does it meet the acceptance criteria? Find the input or data edge case that breaks it (NULLs, duplicates, ties, empty groups, time boundaries).
2. Performance: is it sargable, and what will the plan and reads look like at 10× the data?
3. Readability and team conventions.
4. Verdict: Ship / Ship with changes / Rework.

Don't rewrite the whole thing unless asked. Point to the line, say why, and suggest the change. Give the full reference solution only when I say "Reveal".

Prompt E — Recognition Drill (`Drill: M[n]` or `Drill: mixed`)

Rapid-fire drill for the operations-to-tool habit.

- 8 plain-English requirements or symptoms, one at a time.
- For each, I answer: the tool/feature/pattern, why, and one alternative + the condition where the alternative would win.
- Grade each answer (✓ / ! partial / ✗) with one line of correction, then give the next one.
- "mixed" draws from all completed modules, weighted toward my past misses.
- End with a score and the 2 weakest areas to revisit.

Prompt F — Consolidate (`Consolidate: [topic id]`)

Produce the clean, PDF-ready version of topic [id] from this chat.

- Start from the original Gen response and merge in every clarification from Doubt:/SEEBA exchanges on this topic into the section it belongs to.
- Don't re-explain from scratch, don't summarise, and don't drop content.
- Include my lab attempts only if I say so. Otherwise keep only the problem statements.
- Output as Markdown. It will also be committed to the repo as Database/SQL_Server_Learning_Path/Modules/M[n]/[id]-[slug].md

Prompt G — Module PDF (`Module PDF: M[n]`)

Build one PDF of every consolidated topic in module M[n], in roadmap order. Content: every word of the consolidated topics. No summarising, no questions or prompts from me.

Format:

- A4, 2cm margins, page numbers bottom-centre only.
- Title page: "SQL Server – Module M[n]: [name]", topic list, date.
- Topic header: bold 15pt, then "Difficulty | App-dev | Interview" line.
- Section headers bold 12.5pt, body 10pt Helvetica.
- Code: Courier 8.5–9pt on light-grey background. Validate every code line ≤ 86 chars before rendering. No text cut off anywhere.
- Tables: ReportLab Table with Paragraph cells so text wraps.
- Page break before each topic.

Build with Python + ReportLab in one script (split into parts and merge with pypdf if too long). Name: SQL_M[nn]_[Module_Name].pdf. After building, report the page count, confirm every topic is present, and spot-check one page visually.

If the build is cut off: continue from where it stopped, don't rebuild the finished parts, and merge.

Situational prompts (adapted from the Claude Learning Prompts Guide)

- **Why first:** "Why does [feature] exist? What was the query or workaround before it, and what pain did it remove?"
- **Compare:** "[X] vs [Y]: side by side on RetailOrderDb, and the exact condition where each wins." Examples: CTE vs temp table, EXISTS vs JOIN, RCSI vs NOLOCK.
- **Debug with me:** "Here's my query, the result I expected and what I got. Don't fix it. Ask me what to check first."
- **Performance:** "Here's the plan / STATISTICS IO output. Walk me through reading it top-right to bottom-left, and tell me which operator to look at first."
- **Level up:** "I can write [X]. What does a senior engineer know about it that I don't yet?"
- **Connect:** "How does [topic A] change the way I should think about [topic B]?"
- **Stuck:** "I've been stuck on [ticket] for [N] min. Here's what I tried. Give me the smallest next hint only."
- **Weekly review:** "I covered [topics] this week. Quiz me on them. Weight the questions toward recognition, not syntax."